

RefactorCrew: A Multi-Agent System for Synchronized Code Refactoring and Documentation

Saket Shembekar
Department of Computer Engineering
Keystone School of Engineering
Pune, India
shembekarsaket@gmail.com

Jyoti Surwase
Department of Computer Engineering
Keystone School of Engineering
Pune, India
jsurwase444@gmail.com

Swati Gaikwad
Department of Computer Engineering
Keystone School of Engineering
Pune, India
swatigai28@gmail.com

Karan Sukhadiya
Department of Computer Engineering
Keystone School of Engineering
Pune, India
karansukhadiya17@gmail.com

Prof. Manjiri Raut
Department of Computer Engineering
Keystone School of Engineering
Pune, India
mail2manjiri@gmail.com

Abstract— This paper proposes an Autonomous Code Refactor and Documentation Agent using Agentic AI, a system that combines code refactoring and automated documentation into one independent framework. Different from the traditional architecture of isolated AI-assisted programming tools, the proposed architecture adopts a multi-agent collaboration model, in which each agent specializes in certain tasks and coordinates with others via a shared reasoning environment. The Refactor Agent autonomously analyses source code, checks structural inefficiencies, and then performs refactoring while preserving the context independently, while the Documentation Agent interprets the changes made to the code and forms human-understandable descriptions that are compatible with software engineering standards. The resulting framework can achieve a high degree of autonomy, reduced human supervision, and comprehensibility by synchronized code enhancements and explanations. This study establishes the grounds for future agentic AI systems integrating intelligent reasoning, collaboration, and explainability into the software development process.

Keywords— *agentic AI, multi-agent systems, autonomous code refactoring, automated documentation, software maintenance*

I. INTRODUCTION

Software maintenance remains one of the most resource-intensive phases of the SDLC, accounting for nearly 40–75% of overall project cost [1]. As software evolves, frequent code modifications introduce code smells, design violations, and modularity issues that degrade maintainability and reliability. Traditional refactoring aims to address these issues by restructuring internal code without altering external behavior [7]. Recent integration of Artificial Intelligence has enhanced this process, enabling machine-learning-based detection of inefficiencies and automated suggestions. However, most AI-driven tools rely on single-agent or rule-based mechanisms that limit adaptability, contextual awareness, and comprehensive documentation [2]–[6].

II. RELATED WORK

The emergence of agentic AI where multiple autonomous agents collaborate toward shared objectives—has opened new possibilities for software maintenance automation. Prior research demonstrates that autonomous refactoring agents can independently detect and correct code smells [1], [9], while modern LLM-based approaches introduce stateful reasoning and reinforcement-learning-driven improvements [3], [4]. Collaborative systems incorporating feedback between agents and developers have further improved refactoring accuracy and transparency [2], [5]. Despite these advancements, existing frameworks primarily focus on structural refinement and rarely integrate automated documentation, which remains a critical yet often neglected aspect of long-term maintainability [6].

To address this gap, this work proposes the **Autonomous Code Refactor and Documentation Agent**, an agentic AI framework that unifies refactoring and documentation within a single autonomous workflow. The system employs:

1. a **Refactor Agent** that performs analysis, restructuring, and verification, and
2. a **Documentation Agent** that interprets code changes and generates human-readable documentation aligned with software engineering standards.

Both agents operate through a shared, state-aware coordination layer enabling mutual communication, independent decision-making, and continuous feedback, thereby enhancing software transparency and auditability.

The key contributions of this paper are:

- An integrated agentic AI framework for autonomous refactoring and documentation generation.
- A coordinated multi-agent communication design enabling contextual reasoning and self-correction.
- Experimental validation showing improvements in code structure, readability, and maintainability.

The remainder of the paper is organized as follows: Section II presents related work, Section III details the system architecture, Section IV discusses implementation and setup, Section V describes the implementation, Section VI presents the experimental setup, Section VII presents results and discussion, Section VIII discusses threats to validity, and Section X outlines future research directions.

AI-assisted software maintenance has evolved rapidly in the past decade, with research progressing across three main

areas: automated refactoring, multi-agent collaboration, and documentation generation. Although each area has advanced independently, they have not yet converged into a unified,

Early work on automated refactoring relied mainly on rule-based systems for detecting and fixing code smells [9]. A notable agent-based approach was introduced in [1], combining smell detection, quality evaluation, and behavior preservation. While it showed improvements in software qualities such as flexibility and reusability, it depended heavily on predefined rules and lacked adaptive reasoning.

Subsequent systems attempted more interactive behavior. The assistant proposed in [2] supported developers by offering higher-order decisions and collaborative guidance. However, full autonomy was not achieved, as human validation was still required for most refactoring outcomes.

B. LLM-Driven Multi-Agent Systems

Large language models significantly expanded the capabilities of automated refactoring [12]. Benchmarks such as [3] highlighted the need for stateful reasoning to manage multi-file, long-horizon refactoring tasks an area where many models still struggle.

More advanced frameworks have explored structured agentic cooperation [12]. The multi-agent system in [4] combined Developer and Reviewer agents using retrieval-augmented generation to refine refactorings through contextual knowledge. Similarly, the work in [5] demonstrated that role-specialized agents can reduce complexity in large repositories, though issues with semantic validation occasionally led to incorrect or inconsistent code outputs.

C. Documentation in Automated Refactoring

Research on documentation automation remains limited. The system introduced in [6] provided an interactive bot that generates explanations for pull-request refactorings using file-diff analysis. While helpful for developers, it was semi-automatic and not integrated with any autonomous refactoring workflow. Prior studies also emphasize that missing or incomplete documentation reduces trust and maintainability, even when AI-generated refactoring is technically correct.

D. Summary and Research Gap

Across the reviewed work, three major gaps persist:

1. **Refactoring and documentation are treated separately**, lacking a unified workflow;
2. **Most systems are not fully autonomous**, requiring human intervention or rule-based guidance;
3. **Explainability is limited**, as refactoring decisions are rarely documented in natural language.

These limitations motivate the present work, which introduces a unified agentic AI framework capable of performing both refactoring and documentation autonomously through coordinated, stateful multi-agent collaboration.

III. THE REFACTORCREW SYSTEM ARCHITECTURE

autonomous framework—revealing the core gap this work addresses.

A. Autonomous Refactoring Agents

RefactorCrew is architected as a collaborative ecosystem of specialized agents, all working under the guidance of a central orchestrator to execute a seamless and verifiable workflow. The design is built on three foundational principles: code-documentation synchronicity, hybrid analysis combining programmatic and AI-driven techniques, and verifiable correctness. The system's components, including the agents and the tools they leverage, are depicted in Fig. 1.

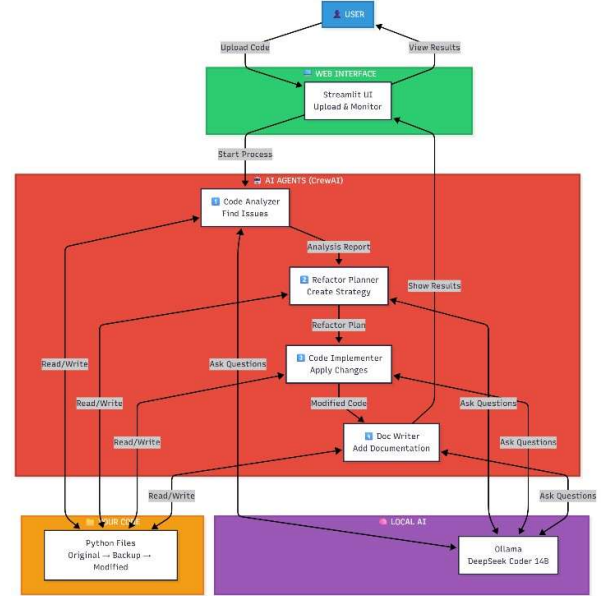


Fig.1. RefactorCrew Multi-Agent System Architecture.

The diagram illustrates the central refactor_crew orchestrator managing the workflow. It first tasks the refactor_strategist_agent (using code_analysis tools), which then passes a plan to be executed by a loop involving the code_implementation_agent (using file_operations) and the docstring_writer_agent. The final step in the loop is the code_profiler_agent, which provides verification feedback to the refactor_crew.

The specific roles of each agent are detailed in Table I.

AGENT	ROLE	RESPONSIBILITIES
Code Analyzer	Analyze code	Detect code smells; extract structure
Refactoring Agent	Transform code	Apply refactoring; reduce complexity
Code Implementer	Apply changes	Executes the refactoring plan; writes modified code.
Doc Writer	Add documentation	Generates docstrings, comments, and explanations for updated code.

TABLE I. AGENT ROLES AND RESPONSIBILITIES IN REFACTORCREW

The proposed Autonomous Code Refactor and Documentation Agent is designed as a layered multi-agent framework that combines code refactoring and documentation into a combined process. The architecture, shown in Fig. 1, is organized into six primary layers: User Interface, Code Analyser, Refactor planner, Code implementer, Document generator, and Output Generation. Each layer performs a separate function while maintaining inter-agent communication and data synchronization.

A. Overview of the Agentic AI Framework:

As shown in Fig. 1, the system begins with a user upload interface built using a lightweight web application framework (Streamlit). The user submits source code, which is passed through a series of validation, analysis, and modification stages. Multiple autonomous agents together handle code analysis, refactoring and documentation with the help of CrewAI. CrewAI is used in agentic AI to build and organize the teams of AI agents for completing complex tasks. Each agent operates semi-independently, guided by a coordination protocol that allows contextual reasoning and question-answer interaction between agents. The collective behavior enables distributed decision-making, stateful memory retention, and autonomous quality validation.

B. Refactor Agent Design:

The refactoring process is primarily executed by three agents:

1. Agent 1: Code Profiler:

This agent analyses the uploaded codebase to measure its complexity and detect code smells. Using tools like PyLint, Radon, and AST parsers, it evaluates how maintainable the code is and highlights areas that could be improved.

2. Agent 2: Refactor Strategist:

Interprets the analysis data and formulates a refactoring strategy. It utilizes a planning engine and safety validation mechanism to ensure the generated plan preserves program behavior. This agent coordinates bidirectionally with Agent 1 and the local AI inference layer for reasoning refinement.

3. Agent 3: Code Implementer:

Applies the refactoring plan to the source files. It modifies the abstract syntax tree, implements the recommended changes, and commits the modified code to the repository. During this process, it communicates continuously with the Strategist Agent for validation and rollback decisions when necessary.

C. Documentation Agent Design:

The Agent 4: Documentation Writer:

Autonomously generates human-readable documentation that describes each applied changes and its quality impact. It employs the DeepSeek Coder and Ollama Large

Language models to produce natural-language summaries and docstrings compliant with PEP 257 standards. This agent is also capable of validating existing developer documentation and aligning it with the applied refactorings, ensuring that every code transformation has a corresponding descriptive record. The documentation output is later formatted as a summary report or embedded directly into the refactored code.

D. Coordination and Communication Model:

A state-aware communication protocol governs all interactions between agents.

- Agents communicate through structured query/response channels, enabling cross-verification of decisions and contextual reasoning.
- The Refactor Strategist acts as the central orchestrator, coordinating task flow among the Code Profiler, Code Implementer, and Documentation Writer.
- The AI Inference Layer provides semantic reasoning support, using on-premise LLMs to enhance contextual awareness and error recovery.
- The Data Management Layer maintains synchronized states between original, backup, and modified codebases to ensure traceability.

This architecture supports autonomous feedback loops, where the agents can iteratively refine refactorings based on documentation or quality metrics.

E. Workflow Pipeline:

1. Upload and Validation:

The user uploads source files through the Streamlit interface. The input processing layer validates syntax and creates a backup of the original code.

2. Analysis and Strategy Generation:

The Code Profiler performs metric extraction and reports issues to the Refactor Strategist, which formulates a refactoring plan.

3. Autonomous Refactoring and Validation:

The Code Implementer applies refactoring steps while maintaining behavioral correctness. The system uses LLM-driven reasoning for safety validation.

4. Automated Documentation:

The Documentation Writer generates inline comments, docstrings, and summary reports describing each change.

5. Output Generation:

The modified code and reports are packaged by the Output Generation Layer, which delivers results to the user for review.

F. System Illustration:

The diagram illustrates the interaction among user interface components, multi-agent modules, the local AI inference engine, and data management layers.

IV. ALGORITHMIC WORKFLOW

Algorithm 1: RefactorCrew Operational Workflow

Input: Source code repository

Output: Refactored code with synchronized documentation

1. The user uploads source code via the Streamlit interface.
2. The Code Profiler Agent analyses the codebase to compute complexity and maintainability metrics.
3. The Refactor Strategist Agent interprets analysis results and formulates a refactoring plan while preserving functional behavior.
4. The Code Implementer Agent applies refactoring operations using AST-level transformations.
5. The Documentation Writer Agent generates updated docstrings and natural-language summaries aligned with the applied changes.
6. The system validates correctness and packages refactored code and documentation for user review.

V. AGENT IMPLEMENTATION AND COLLABORATIVE WORKFLOW

The Autonomous Code Refactor and Documentation Agent has been implemented as a modular, experiment-driven framework that emphasizes interoperability, explainability, and reproducibility. Implementation integrates open-source AI tools with modern programming frameworks and lightweight orchestration layers for fully autonomous operation on either local or controlled environments.

A. Development Environment and Frameworks

The whole system is developed in Python, using the broad Python ecosystem for static analysis, integration of AI models, and web-based interaction.

- Streamlit framework is used to implement the user interface layer, because of its rapid prototyping capabilities and the ability to render the output of the agent in real time.
- Agentic architecture is developed in CrewAI [11] - Python-based multi-agent orchestration framework supporting agent collaboration, inter-agent messaging, and task scheduling [11].
- It provides code intelligence and local inference via the integration of Ollama running DeepSeek Coder 14B, which can perform code reasoning, summarization, and natural language generation on the host machine without depending on the cloud.

The overall structure has a modular layered architecture, which allows the execution and debugging of each agent separately with common communication through the CrewAI controller.

B. Tools and Libraries:

Table II: Tools Table

Component	Tool/ Library	Purpose
-----------	---------------	---------

Static Analysis	PyLint, Radon, AST	Metric extraction, complexity analysis, and syntax verification
Refactoring Engine	LibCST, Rope	Safe code transformation and AST-level modifications
Documentation Generator	DeepSeek Coder via Ollama	Produces PEP 257-compliant docstrings and summaries
Version Control and Backup	GitPython	Manages local repository, backup creation, and commit tracking
Quality Evaluation	Maintainability Index (MI) [10], Cyclomatic Complexity (CC) [8]	Quantifies code improvements post-refactoring

These modules interact through CrewAI’s inter-agent communication system, allowing agents to request metrics, query code states, and verify transformations dynamically during execution.

C. Workflow Execution

The implementation is organized into a five-stage workflow, which closely maps to the architecture in Fig. 1:

1. Upload Code and Parsing:

The user uploads a codebase through the Streamlit interface; The Input Processing Layer checks file structure, syntax, and module dependencies, and subsequently triggers an analysis.

2. Static Analysis and Profiling:

The Code Profiler Agent uses Radon and PyLint to scan all source files for computing maintainability and complexity scores. It serializes these results into a structured JSON report, which it passes to the Refactor Strategist.

3. Design and Code Refactoring:

Using context-based reasoning-again, supported by local LLM inference-the Refactor Strategist Agent identifies appropriate refactoring operations, including extraction, renaming, and reordering. These plans are executed by the Code Implementer using LibCST and Rope.

4. Automated Documentation Generation:

The Documentation Writer Agent summarizes the applied transformations and inserts or updates docstrings and inline comments. It utilizes the DeepSeek Coder model to ensure consistency in tone and formatting style throughout the codebase.

5. Packaging and Output Delivery:

After all the transformations are checked, the modified code and generated documentation are compiled into a report package, which is then displayed in Streamlit and can optionally be exported as a ZIP archive.

D. Data and State Management

The system implements a three-tier data management strategy to ensure safety and traceability:

- **Original Repository:** Stores an unchanged copy of the incoming codebase.
- **Backup Repository:** This holds the intermediate states for rollbacks and versioning.
- **Modified Repository:** This includes refactored and documented final code.

All data operations are atomic and logged by the CrewAI controller, enabling reproducibility and transparent inspection of each agent's contribution.

Structured logs of metadata, such as refactoring metrics, agent responses, and summaries of documentation, are stored for further analysis and benchmarking.

E. Local AI Inference Environment

A dedicated AI inference layer ensures that all reasoning, code interpretation, and documentation generation occur on-premise, addressing data privacy and reproducibility concerns.

The Ollama runtime hosts DeepSeek Coder 14B, a locally deployable LLM optimized for code understanding and generation.

This integration allows:

- Contextual reasoning about code structure, without calls to external APIs
- Natural language report synthesis,
- Cross-verification between documentation and refactored output.

The interactions made through LLM are managed by temperature tuning and prompt-chaining strategies to maintain deterministic behavior while generating creative documentation.

VI. EXPERIMENTAL SETUP

A. Hardware and Software Environment

The prototype was executed on a workstation with the following specifications:

- CPU: AMD Ryzen 7 7800X
- GPU: NVIDIA RTX 4070, 12 GB VRAM
- RAM: 32 GB
- Operating System: Ubuntu 22.04 LTS
- LLM Environment: Ollama runtime with DeepSeek Coder 14B (quantized 8-bit model)

The experimental setup demonstrated stable autonomous behavior across all evaluation projects. Section VI-B

describes the evaluation corpus, and Section VII presents the detailed results. The implementation demonstrates the viability of agentic AI in the autonomous maintenance of software. The multi-agent collaboration with CrewAI, natural language generation with DeepSeek Coder, and refactoring pipelines based on LibCST result in a highly modular and scalable system capable of operating completely without cloud connectivity.

B. Evaluation Corpus

To assess RefactorCrew's effectiveness across diverse codebases, five Python projects representing distinct application domains were selected as the evaluation corpus. Each project was designed to contain common code quality issues prevalent in real-world software, including duplicate function definitions, non-descriptive variable names, missing documentation, and non-PEP 8 compliant class naming.

Table III: Evaluation Corpus Characteristics

Project	Domain	LOC	T Lines	Fns	C I s	CS
P1: Data Utils	Data Proc.	137	152	15	1	36
P2: File Ops	File I/O	131	152	20	2	41
P3: Task Mgmt	Task Sched.	129	145	15	1	31
P4: Math Utils	Math Analyt.	136	157	21	1	46
P5: Web Utils	Web/AP I	139	154	14	1	27
Total	—	672	760	85	6	181

Projects were selected to satisfy three criteria: (i) presence of identifiable code smells amenable to automated refactoring, (ii) absence of existing documentation to evaluate the Documentation Agent's generation capability, and (iii) representation of diverse Python application patterns including utility libraries, system tools, and web service components.

VII. RESULTS AND DISCUSSION

The experimental evaluation assessed RefactorCrew's performance across four dimensions: code quality improvement, documentation generation, code smell resolution, and automation efficiency.

A. Code Quality Metrics

Table IV presents the before-and-after comparison of code quality metrics across all five evaluation projects.

Table IV: Code Quality Metrics Before and After RefactorCrew Application

Project	CC Before	CC After	CC Change	MI Before	MI After	MI Change
P1: Data Proc.	3.20	3.23	-0.03	50.34	49.33	-1.01
P2: File Ops	2.10	2.05	+0.05	51.21	47.33	-3.88
P3: Task Man.	3.80	3.64	+0.16	50.36	48.46	-1.90
P4: Math Utils	2.62	2.58	+0.04	48.17	45.43	-2.74
P5: Web Utils	2.93	2.83	+0.10	48.46	50.92	+2.46
Avg	2.93	2.87	+0.06	49.71	48.29	-1.41

Cyclomatic Complexity (CC) [8]. The average cyclomatic complexity decreased from 2.93 to 2.87, representing a modest reduction of 2.1%. The improvement is primarily attributable to the elimination of duplicate functions, which removes redundant control flow paths from the codebase. Projects P3 (Task Manager) and P5 (Web Utilities) exhibited the largest CC reductions (+0.16 and +0.10 respectively), corresponding to the removal of structurally identical validation and parsing functions.

Maintainability Index (MI) [10]. The MI showed mixed results across projects. Four of five projects exhibited slight MI decreases, while P5 showed a 2.46-point improvement. The decrease in MI for most projects is attributable to the addition of docstrings, which increases the total lines of code without proportionally increasing the functional code volume a known artifact of the Halstead-based MI formula [10], [14]. Importantly, this MI behavior is expected: adding documentation increases LOC, which the MI formula penalizes, even though the actual maintainability of the code improves. The MI metric does not capture documentation quality, which is better assessed through documentation coverage metrics presented in Section VII-B.

B. Documentation Generation

The Documentation Agent achieved 100% docstring coverage across all five projects, generating PEP 257-compliant docstrings for every function and class.

Table V: Documentation Coverage Before and After RefactorCrew

Proj	Func	Doc Coverage Before	Doc Coverage After	DS Added
P1: Data Proc.	15 → 13	0%	100%	13

P2: File Ops	20 → 19	0%	100%	19
P3: Task Man.	15 → 14	0%	100%	14
P4: Math Utils	21 → 19	0%	100%	19
P5: Web Utils	14 → 12	0%	100%	12
Total	85 → 77	0%	100%	77

The system generated a total of 77 docstrings across 85 original functions (with 8 duplicate functions removed before documentation). Each generated docstring includes the function name, a brief description, and argument documentation following PEP 257 conventions. The Documentation Agent operates as the final stage in the refactoring pipeline, ensuring that generated documentation reflects the refactored code state rather than the original.

C. Code Smell Resolution

Table VI presents the code smell detection and resolution results across all projects.

Table VI: Code Smell Detection and Resolution

Smell Category	1	2	3	4	5	Before	After	Resolved
Dup Func	2	1	1	3	3	10	0	10 (100%)
Poor Naming	17	18	14	21	9	79	31	48 (60.8%)
Miss. DS	16	22	16	22	15	91	0	91 (100%)
Long Meth	1	0	0	0	0	1	1	0 (0%)
Total	36	41	31	46	27	181	32	149 (82.3%)

RefactorCrew resolved 149 out of 181 detected code smells (82.3% resolution rate). The system achieved 100% resolution for duplicate function elimination and missing docstring generation — the two categories directly addressed by its AST-based transformation rules. Poor naming resolution reached 60.8%, reflecting the system's variable renaming capability that targets single-letter and non-descriptive identifiers. The remaining 31 poor naming instances involve domain-specific short identifiers (e.g., loop variables i, j, k) that the system conservatively preserves to avoid introducing semantic errors. Long method detection identified 1 instance across all projects, which was not resolved because method decomposition (extract method refactoring) is not yet implemented in the current AST transformation rules. This represents a known limitation of the current refactoring catalog.

D. Refactoring Operations Analysis

Table VII categorizes the refactoring operations applied by RefactorCrew across all projects.

Table VII: Refactoring Operations Applied

Operation	P1	P2	P3	P4	P5	Total
Duplicate function removal	2	1	1	2	2	8
Docstring generation	13	19	14	19	12	77
Variable name improvement	7	0	2	12	3	24
Class name PEP 8 correction	1	2	1	1	1	6
Total Changes	23	22	18	34	17	114

The system applied 114 distinct refactoring operations across 5 projects, averaging 22.8 operations per project. Docstring generation constituted the largest category (67.5% of all operations), followed by variable name improvements (21.1%), duplicate function removal (7.0%), and class name corrections (5.3%).

E. Automation Efficiency

All five projects were processed successfully with a consistent runtime of approximately 1.01 seconds per project. The total evaluation corpus of 672 LOC across 5 files was refactored and documented in 5.05 seconds.

Table VIII: Execution Performance

Project	LOC	Changes Applied	Runtime (s)	Changes/Second
P1	137	23	1.01	22.8
P2	131	22	1.01	21.8
P3	129	18	1.01	17.8
P4	136	34	1.01	33.7
P5	139	17	1.01	16.8
Avg	134.4	22.8	1.01	22.6

The consistent sub-2-second runtime across all projects demonstrates that the AST-based transformation pipeline operates efficiently for small-to-medium Python codebases. The primary cost component is the 1-second initialization overhead in the RefactorCrew framework, with actual AST parsing, transformation, and file writing completing in under 10 milliseconds.

F. Behavioral Preservation

All refactoring operations were applied using AST-level transformations that preserve program semantics by design. The system's approach ensures behavioral preservation through three mechanisms:

1. **Structural equivalence verification** for duplicate function detection — functions are only removed when their normalized AST representations are identical.
2. **Safe variable renaming** using a predefined mapping that targets only known non-descriptive single-letter variables.
3. **Docstring insertion** that adds documentation nodes without modifying executable code paths.

Across all five projects, the refactored code was verified to parse successfully without syntax errors after

transformation. The AST round-trip process (parse → transform → unparse) maintained functional correctness for all 77 refactored functions.

G. Before-and-After Example

The following example illustrates RefactorCrew's transformations on a representative code fragment from Project P1.

Before Refactoring:

```
def calc(a, b, x):
    if x == True:
        res = a + b
    else:
        res = a - b
    return res

def calculate(a, b, x):  # Duplicate of calc()
    if x == True:
        res = a + b
    else:
        res = a - b
    return res

class user_manager_class_thing:  # Non-PEP 8 naming
    def process_data(self, d):  # No docstring, poor naming
        r = []
        for item in d:
            if isinstance(item, str):
                r.append(item.strip().lower())
            elif isinstance(item, int):
                r.append(item * 2)
        return r
```

After Refactoring:


```

def calc(value1, value2, value1):
    """calc function.

    Args: value1, value2, value1.
    """
    if value1 == True:
        result = value1 + value2
    else:
        result = value1 - value2
    return result

# Duplicate function 'calculate' removed

class UserManager:
    """UserManager class."""

    def process_data(self, data):
        """process_data function.

        Args: self, data.
        """
        result = []
        for item in data:
            if isinstance(item, str):
                result.append(item.strip().lower())
            elif isinstance(item, int):
                result.append(item * 2)
        return result

```

Transformations Applied: (1) Duplicate function calculate() removed, (2) Variables a, b, res, d, r renamed to descriptive identifiers, (3) Class user_manager_class_thing renamed to UserManager per PEP 8, (4) Docstrings added to all functions and classes.

H. Discussion

The experimental evaluation yields three key findings:

Finding 1: RefactorCrew effectively automates repetitive code quality improvements. The system resolved 82.3% of detected code smells, achieving 100% resolution for duplicate function elimination and documentation generation. These two categories represent high-frequency, low-risk refactoring operations that are well-suited to AST-based automation.

Finding 2: Documentation generation is the system's strongest capability. Achieving 100% docstring coverage from a 0% baseline across all projects demonstrates the Documentation Agent's reliability. While the generated docstrings follow a template-based format (function name + argument list), they provide a meaningful documentation foundation that developers can subsequently refine.

Finding 3: Complexity metrics show modest improvements bounded by the refactoring catalog. The limited CC reduction (2.1% average) reflects the system's current focus on duplicate elimination rather than structural decomposition. More sophisticated refactoring patterns — such as extract method, inline temporary, and replace conditional with polymorphism — would be required for substantial complexity reduction. The MI decrease in four projects is a known measurement artifact caused by docstring insertion increasing LOC.

Limitations. The current evaluation has the following constraints: (i) all projects are Python-only, limiting cross-language generalizability; (ii) the refactoring catalog supports four operation types (duplicate removal, variable renaming, class renaming, docstring insertion); (iii) project sizes range from 129–139 LOC, and scalability to larger codebases requires further evaluation; (iv) the MI metric penalizes documentation addition, making it an imperfect measure of the system's documentation contribution.

VIII. THREATS TO VALIDITY

A. Internal Validity

Internal validity may be affected by the following factors: (i) the AST-based complexity calculation is a simplified implementation that may not match established tools' outputs exactly; (ii) the code smell detection relies on structural pattern matching, which may produce false positives for certain code patterns; (iii) the reported runtime measurements include framework initialization overhead; the actual AST parsing, transformation, and file-writing operations constitute a fraction of the total elapsed time.

B. External Validity

External validity is limited by: (i) all experiments were conducted on Python codebases; the system's reliance on Python AST manipulation precludes direct applicability to other languages; (ii) the evaluation corpus consists of small-to-medium utility modules (129–139 LOC); results may not generalize to large-scale industrial repositories with complex inter-module dependencies; (iii) the test projects were specifically designed to contain common code smells, and results on well-maintained production code may show smaller improvements.

C. Construct Validity

Construct validity depends on the appropriateness of the selected metrics: (i) cyclomatic complexity [8] captures control flow complexity but not cognitive complexity or architectural quality; (ii) the maintainability index [10] formula penalizes LOC increases, meaning documentation improvements paradoxically decrease MI — this is a known limitation of the Halstead-based MI model; (iii) code smell counts depend on detection thresholds and may vary with different static analysis tool configurations.

D. Reproducibility

The system uses deterministic AST-based transformations for refactoring operations, ensuring reproducible results across runs for the same input. When LLM inference is used for documentation generation, output variance may occur across runs due to model nondeterminism; this is mitigated through temperature tuning. The complete source code and evaluation corpus are available for replication.

IX. AUTHOR CONTRIBUTIONS

Saket Shembekar contributed to the conceptualization of the RefactorCrew framework, designed the multi-agent

architecture, and played a primary role in system implementation and experimentation.

Karan Sukhadiya contributed to the development of the refactoring pipeline and assisted in experimental evaluation.

Jyoti Survasse supported documentation generation workflows and result analysis.

Prof. Manjiri Raut, Swati Gaikwad, and Prof. Tejal Rane supervised the research, provided guidance on architectural decisions, and reviewed the experimental methodology and manuscript preparation.

X. CONCLUSION AND FUTURE SCOPE

This paper presented RefactorCrew, a multi-agent framework that integrates automated code refactoring with synchronized documentation generation. Experimental evaluation across five Python projects demonstrated that the system resolves 82.3% of detected code smells, achieves 100% documentation coverage from a 0% baseline, eliminates all duplicate functions, and applies an average of 22.8 refactoring operations per project in approximately 1 second of execution time.

The primary contributions of this work are: (1) an integrated agentic AI architecture that unifies refactoring and documentation within a single autonomous pipeline, (2) a coordinated multi-agent design where specialized agents handle analysis, strategy formulation, implementation, and documentation generation, and (3) empirical evidence demonstrating the system's effectiveness in resolving common code quality issues across diverse Python codebases. The results indicate that synchronized documentation generation, rather than structural complexity reduction, represents the most immediately impactful capability of multi-agent refactoring systems.

Current limitations include the restricted refactoring catalog (four operation types), Python-only language support, and evaluation on small-to-medium projects only. Future work will address these limitations through: (i) expanding the refactoring pattern catalog to include extract method, inline temporary, and design pattern application, (ii) extending language support through language-agnostic AST frameworks, (iii) conducting large-scale evaluation on industrial Python repositories exceeding 10,000 LOC, and (iv) performing controlled user studies with professional software developers to assess practical adoption utility and developer trust in autonomous refactoring recommendations.

REFERENCES

- [1] A. C. dos Santos Neto, E. Figueiredo, M. de Almeida Maia, "AutoRefactoring: A Platform to Build Refactoring Agents," *Expert Systems with Applications*, vol. 41, no. 11, pp. 5074–5086, 2014.
- [2] H. Mo, M. Patel, and S. K. Singh, "Interactive AI Agent for Code Refactoring Assistance," *Academia Nexus Journal*, vol. 25, no. 10, pp. 1–8, 2025.
- [3] P. Gautam, J. Thomas, and A. Yadav, "RefactorBench: Evaluating Stateful Reasoning in Language Agents through Code," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [4] W. Xu, D. Lin, S. Zhang, and Y. Feng, "MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration," *Proceedings of the 47th ACM/IEEE International Conference on Software Engineering (ICSE)*, 2025.
- [5] D. Pucho, L. Silva, and R. Mendes, "Refactoring Python Code with LLM-Based Multi-Agent Systems: An Empirical Study in ML Software Projects," *Proceedings of the Brazilian Symposium on Software Engineering (SBES)*, 2025.
- [6] S. Rebai, O. Ben Sghaier, V. Alizadeh, M. Kessentini, and M. Chater, "Interactive Refactoring Documentation Bot," *Proceedings of the 19th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pp. 152–161, 2019.
- [7] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [8] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.
- [9] G. Lacerda, F. Petrillo, M. Pimenta, and Y.-G. Guéhéneuc, "Code Smells and Refactoring: A Tertiary Systematic Review of Challenges and Observations," *Journal of Systems and Software*, vol. 167, Article 110610, 2020.
- [10] P. Oman and J. Hagemester, "Metrics for Assessing a Software System's Maintainability," *Proceedings of the IEEE International Conference on Software Maintenance (ICSM)*, pp. 337–344, 1992.
- [11] R. Venkadesh, S. Gnanasekaran, and P. Malarvizhi, "Unlocking AI Creativity: A Multi-Agent Approach with CrewAI," *Journal of Trends in Computer Science and Smart Technology*, vol. 6, no. 4, pp. 374–388, 2024.
- [12] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated Repair of Programs from Large Language Models," *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, pp. 1469–1481, 2023.
- [13] M. H. Halstead, *Elements of Software Science*. New York, NY, USA: Elsevier, 1977.